More Height-Balanced Trees

B-Trees and B^+-Trees

Saptarshi Pyne, Assistant Professor
Department of Computational and Data Sciences (CDS)
Indian Institute of Science Education and Research (IISER) Kolkata
Mohanpur, West Bengal, India 741246

CS3201 Programming and Data Structures II
March 25, 2025

Recap

In the previous classes, we discussed two height-balanced **binary** search trees, namely, the AVL tree and the red-black tree.

Today

Today we will discuss two height-balanced **m-ary** search trees, namely, the B-tree and the B^+-tree.

Development of the Height-Balanced Binary Search Trees: Recap

Year	Event
Late 1950s	**The binary search tree** was independently proposed by several researchers.
1962	**The AVL tree** was proposed by two Soviet mathematicians Adel'son-Vel'skiˇı (pronounced velskii) and Landis.
1972	**The red-black tree** was proposed by a German computer scientist Rudolf Bayer. At that time, Bayer called it the 'symmetric binary B-trees'.
1978	**The red and black colour convention** was introduced by two American computer scientists Guibas and Sedgewick.

Development of B-Trees

Year	Event
1972	Rudolf Bayer and Edward McCreight, while working at Boeing Research Labs, proposed **B-trees** for managing file indices. It was published in issue 3 of volume 1 of *Acta Informatica* [1].
1972	In issue 4 of volume 1 of *Acta Informatica*, Bayer proposed the 'symmetric **binary** B-trees' [2] that would later be known as **the red-black trees**.
1979	Douglas Comer (at Purdue University) published a survey of B-trees and its variants, including **B^+-trees**, in *ACM Computing Surveys*. Comer mentioned that B+-trees had been in use in IBM's virtual storage access method (VSAM). However, there is no single paper proposing the notion of B^+-trees.

[1] https://link.springer.com/article/10.1007/BF00288683
[2] https://link.springer.com/article/10.1007/BF00289509
[3] https://dl.acm.org/doi/pdf/10.1145/356770.356776

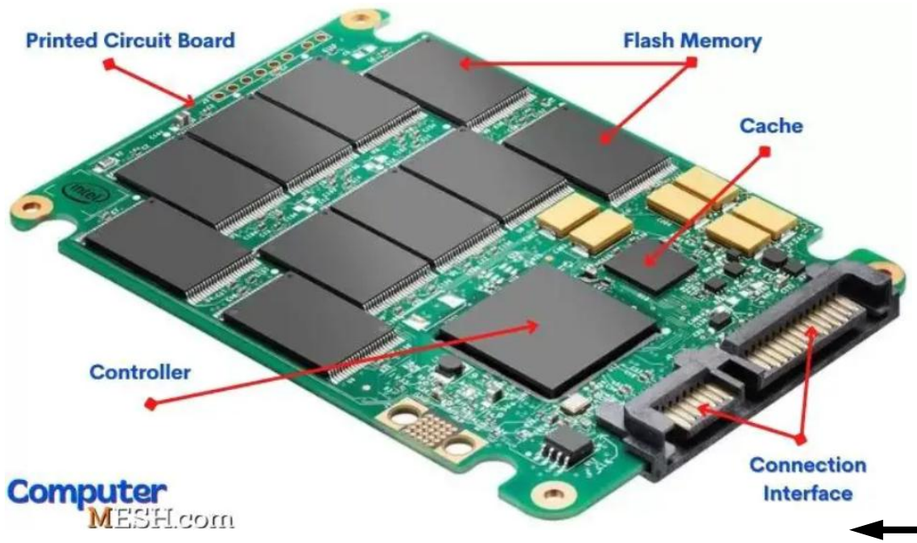
B-Trees

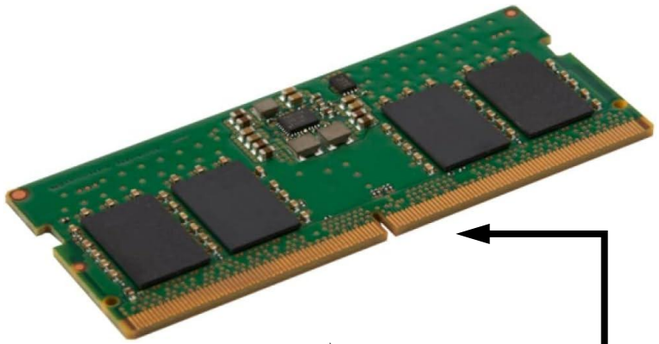
Nobody knows what the ‘B’ in B-tree stands for.
Bayer and McCreight kept it a mystery.

B-Trees

A B-tree is a height-balanced search tree designed to accommodate a **secondary storage**.

Typical Memory Hierarchy in a Computer





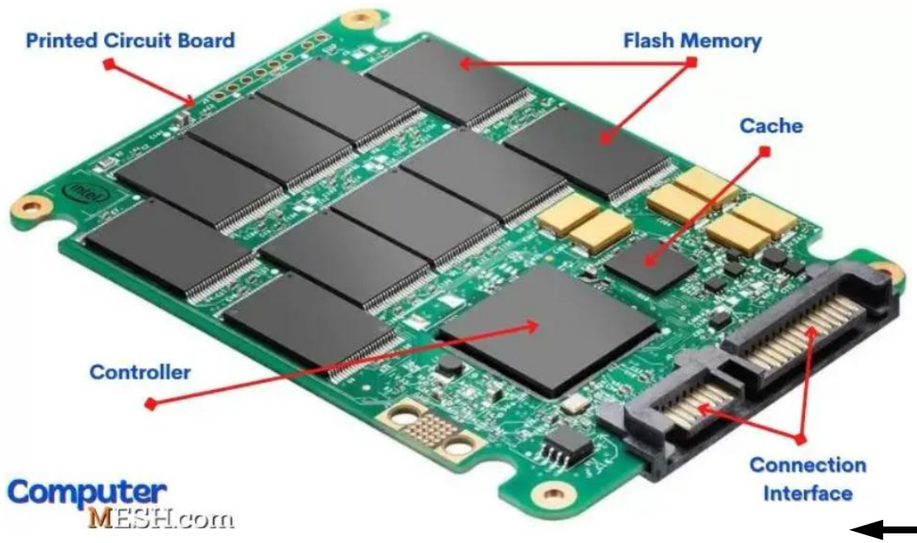
Secondary Storage (disk drive)
A typical NVMe PCIe SSD has a bandwidth of only **3.5 Gbps**. Substantially slower than the main memory.

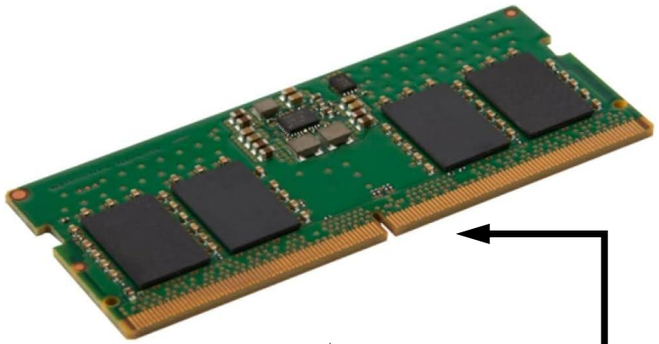
Main Memory (RAM)
A typical DDR5 SDRAM has a bandwidth of **51.2 Gbps**.

CPU

CPU: https://circuitdigest.com/news/dhruv64-debuts-as-india-first-indigenous-10-ghz-64-bit-dual-core-processor
RAM: https://www.amazon.in/HP-DDR5-SDRAM-Memory-Module/dp/B0BSLPTSKD

Typical Memory Hierarchy in a Computer (contd.)





A secondary storage typically has a capacity in TBs. It usually divides its content into 512B **blocks**.

When a block is requested by the CPU, the block is copied from the secondary storage to the main memory. Subsequently, CPU performs reads/writes on that block. If a write operation is performed on the block, the block is copied back to the secondary storage.

CPU

CPU: https://circuitdigest.com/news/dhruv64-debuts-as-india-first-indigenous-10-ghz-64-bit-dual-core-processor
RAM: https://www.amazon.in/HP-DDR5-SDRAM-Memory-Module/dp/B0BSLPTSKD

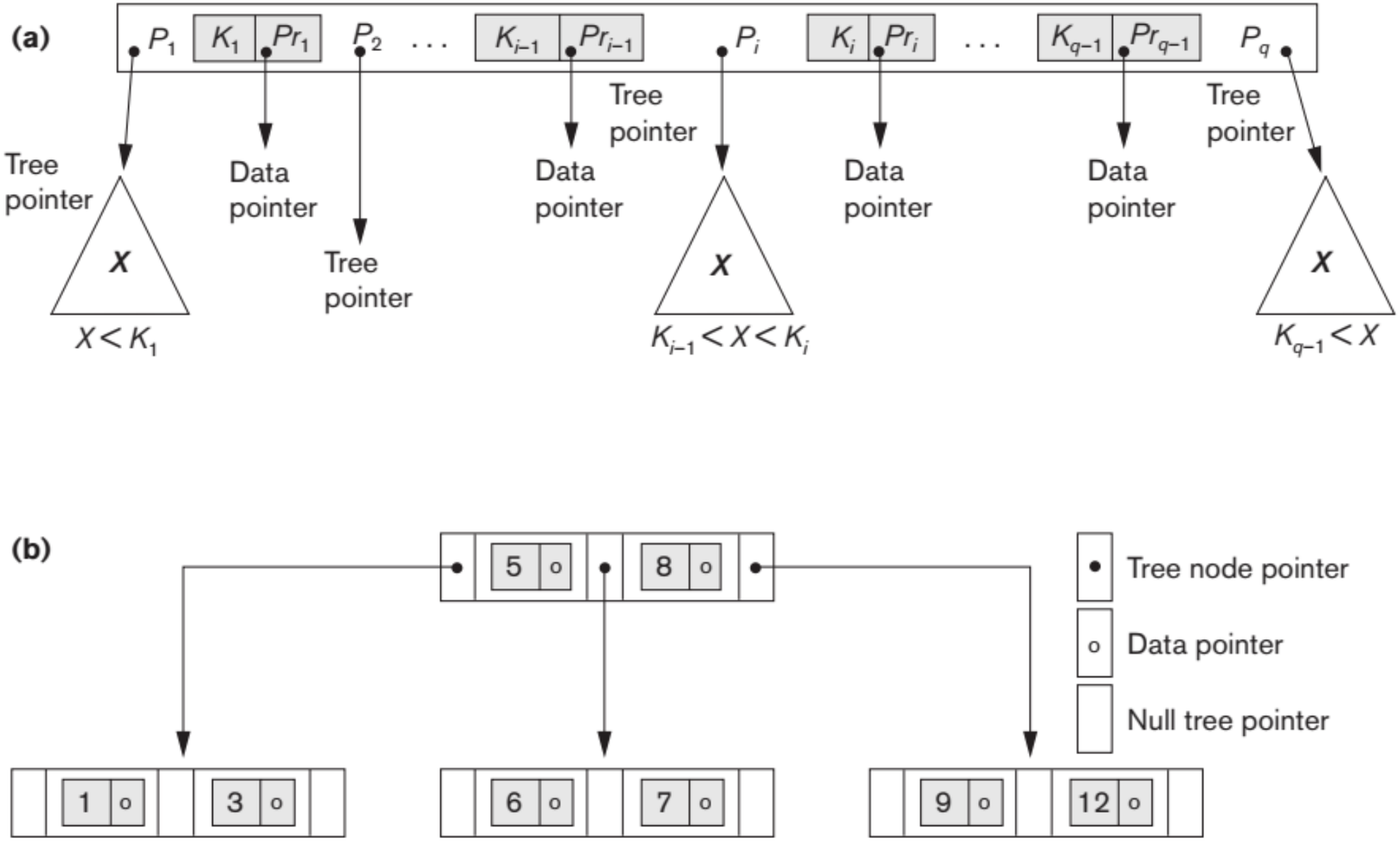
B-Trees (contd.)

Since the secondary storage is substantially slower than the main memory, we should **minimise the number of block transfers to and from the disk**.

If we only keep one key in a node, the search tree would become quite large and would require a large number of blocks.

Hence, a B-tree stores as many keys as possible in a node. Ideally, a node should be the size of a block. Suppose, a block is 512B and each key (along with its satellite data pointer) requires 8B. Then each node can accommodate (512 / 8) = 64 keys at maximum.

B-Trees (contd.)



Here, we assume that each node can have at most 2 keys. In practice, a B-tree node can have 2,000 keys.

Figure 17.10, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

B^+-Trees

B^+-Trees

A B^+-tree is one of the natural variants of the B-trees. It has been widely used for organising file indices.

In a B^+-tree, the **satellite data pointers** are stored only in the **leaf nodes**.

Structures of the B$^+$-Trees (a) Internal and (b) Leaf Nodes

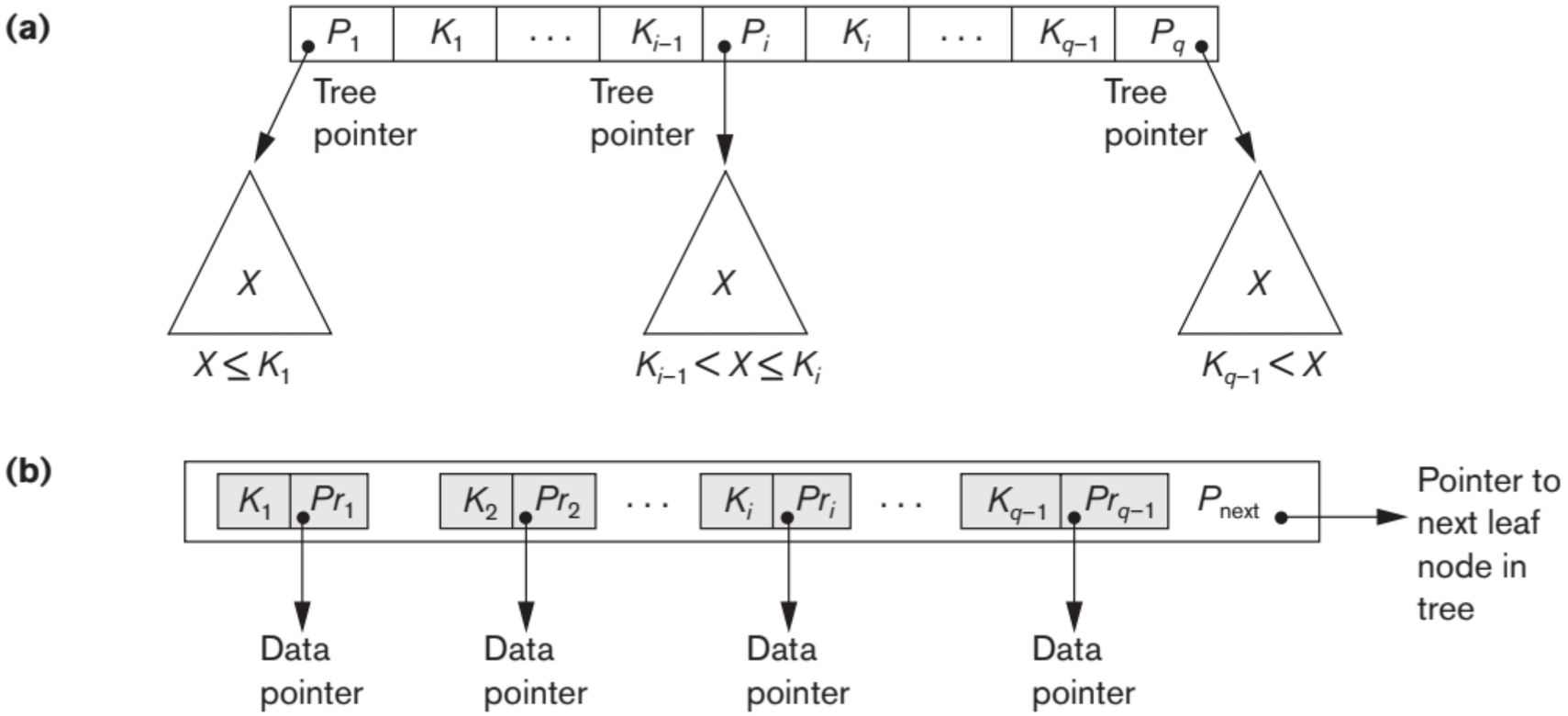


Figure 17.11, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

B^+-Trees: An Example

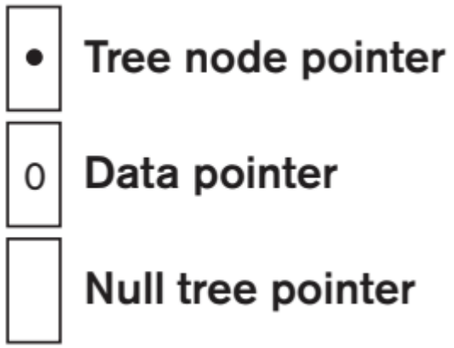
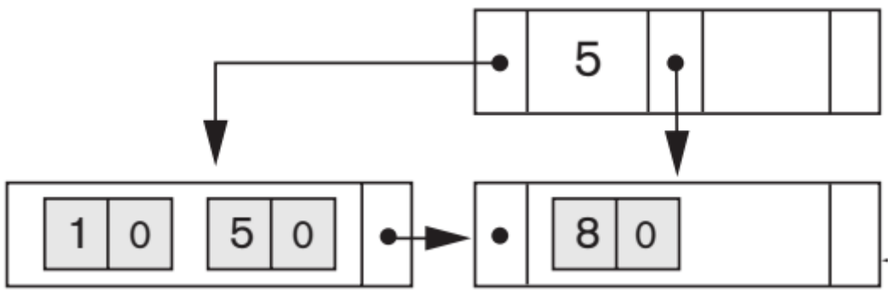


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Insert 1 into the B$^+$-Tree: An Overflow Occurrs



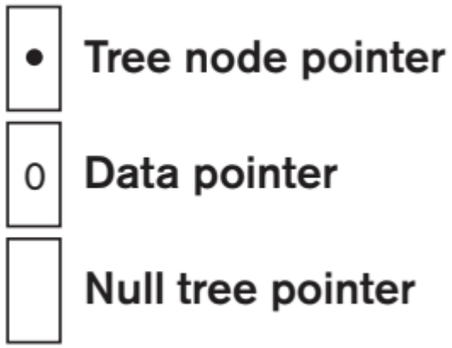
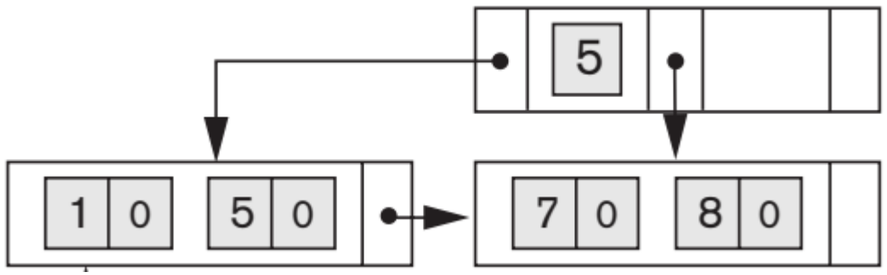


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Insert 1 into the B$^+$-Tree: Insert 7



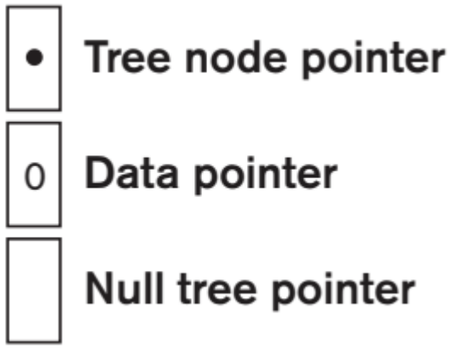


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Insert 1 into the B$^+$-Tree: Insert 3, Overflow

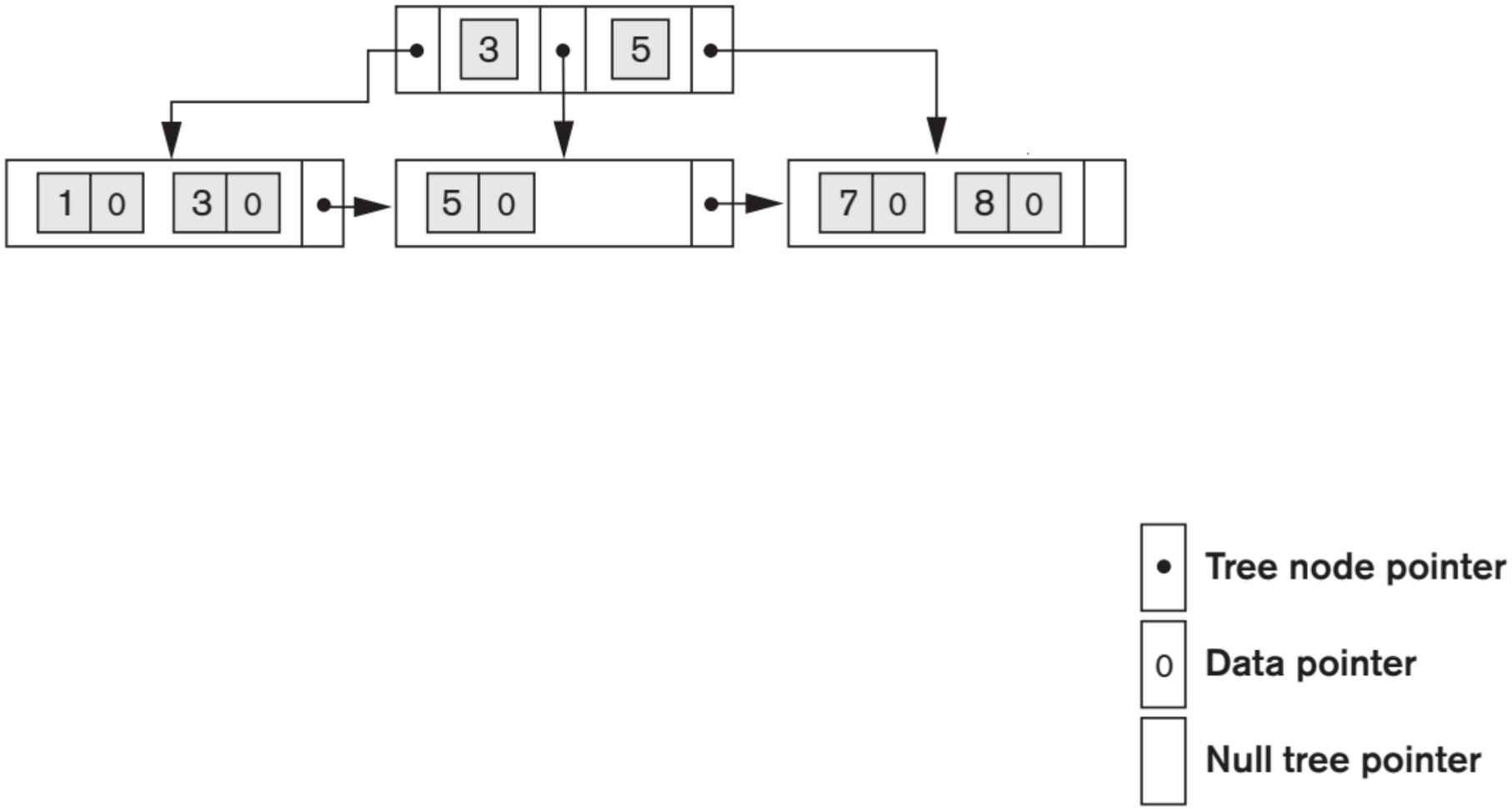


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Insert 1 into the B$^+$-Tree: Insert 12, Overflow

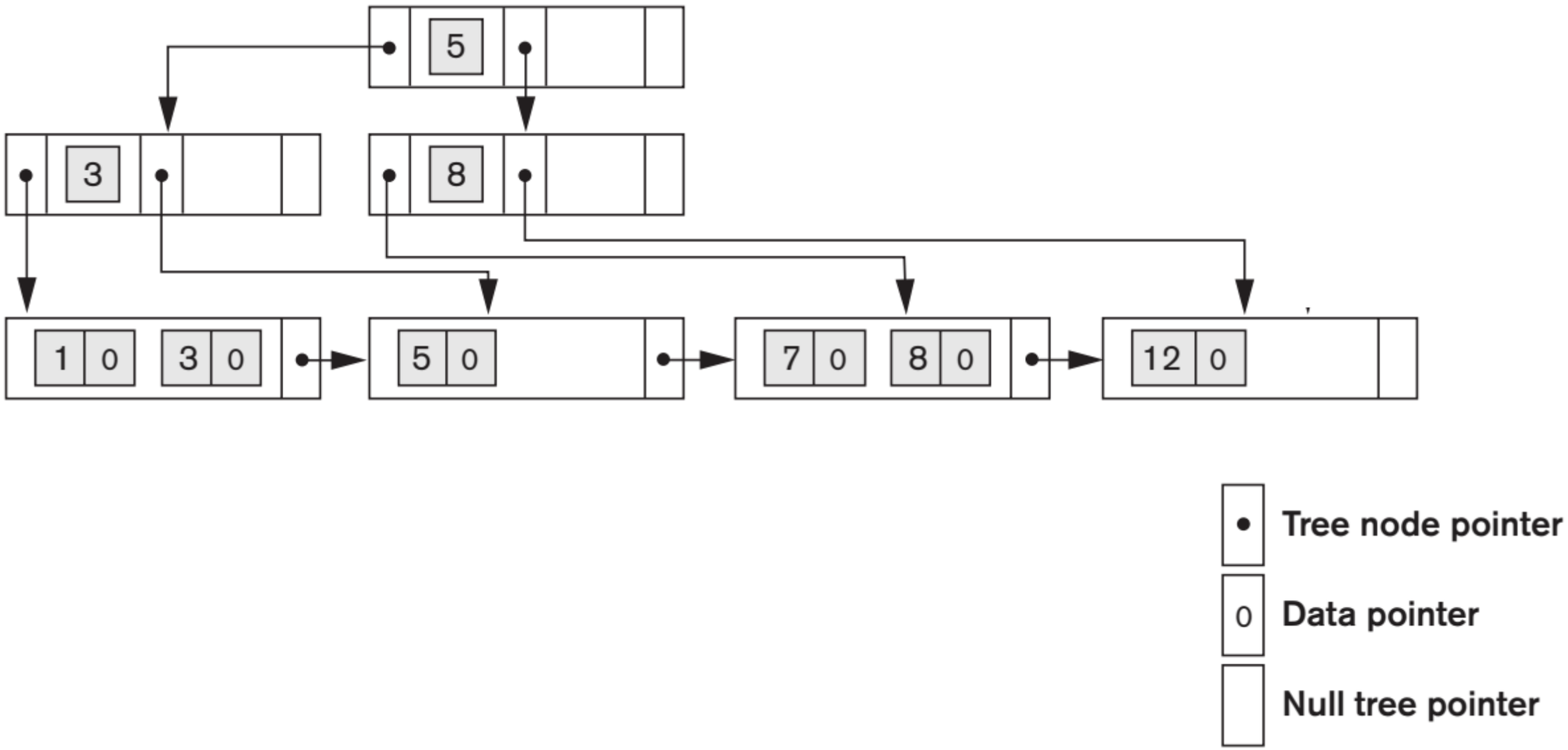
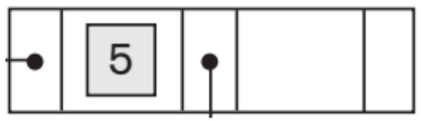


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Searching for 7 in the B^+-Tree



We only keep the root node of the B+-tree in the main memory.

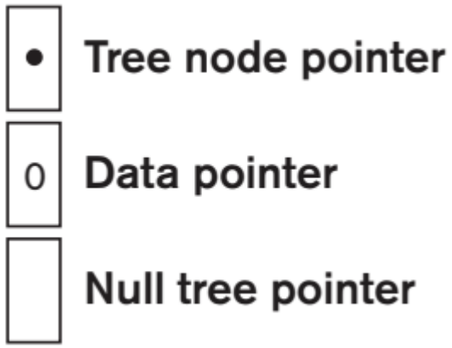
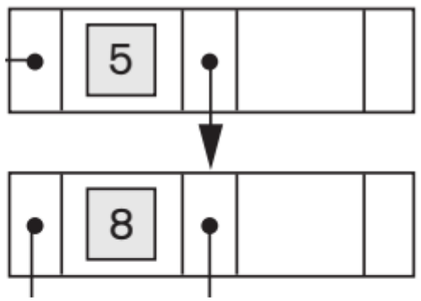


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Searching for 7 in the B$^+$-Tree (contd.)



Since 7 > 5, we bring the node pointed to by 5's right tree pointer from the disk.

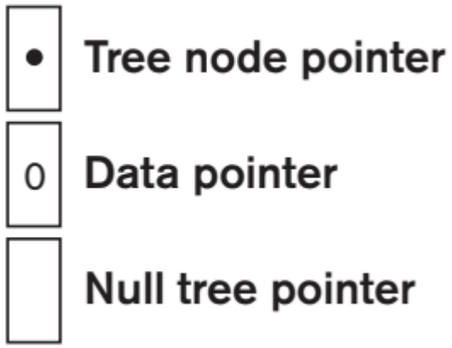
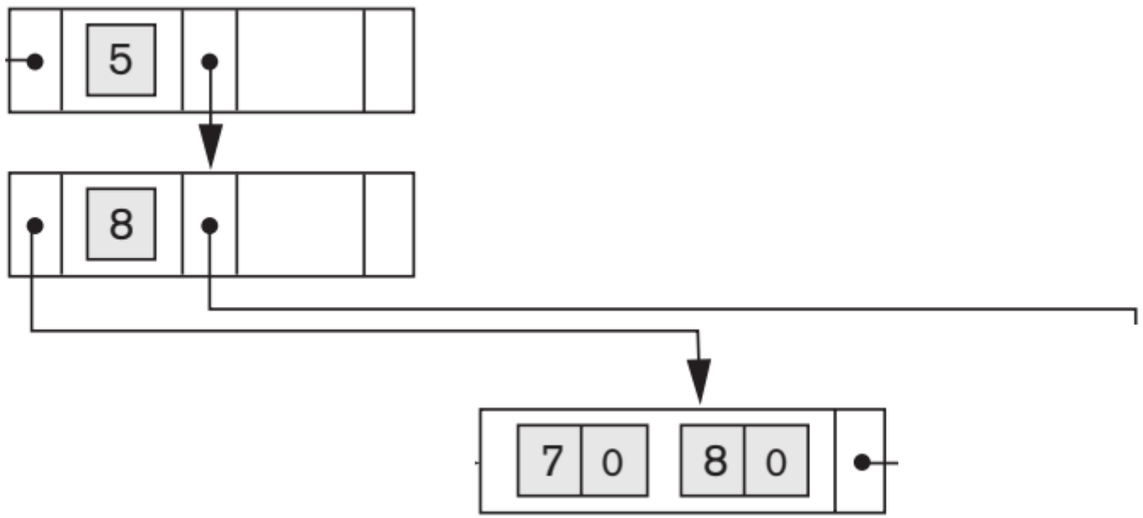
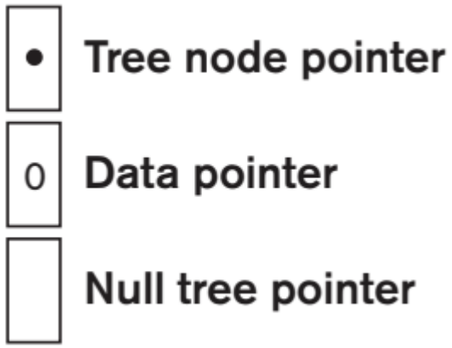


Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Searching for 7 in the B$^+$-Tree (contd.)





Since 7 < 8, we bring the node pointed to by 8's left tree pointer from the disk. Thus, we get access to index 7 and its satellite data pointer in **only 2 disk accesses**. Such a low number of disk accesses is made possible by **keeping the search tree height-balanced** and **simultaneously maximising the number of keys in each node**.

Figure 17.12, Fundamentals of Database Systems, Elmasri and Navathe, 7th Edition, 2016

Summary

Today we discussed two height-balanced **m-ary** search trees, namely, the B-tree and the B^+-tree.

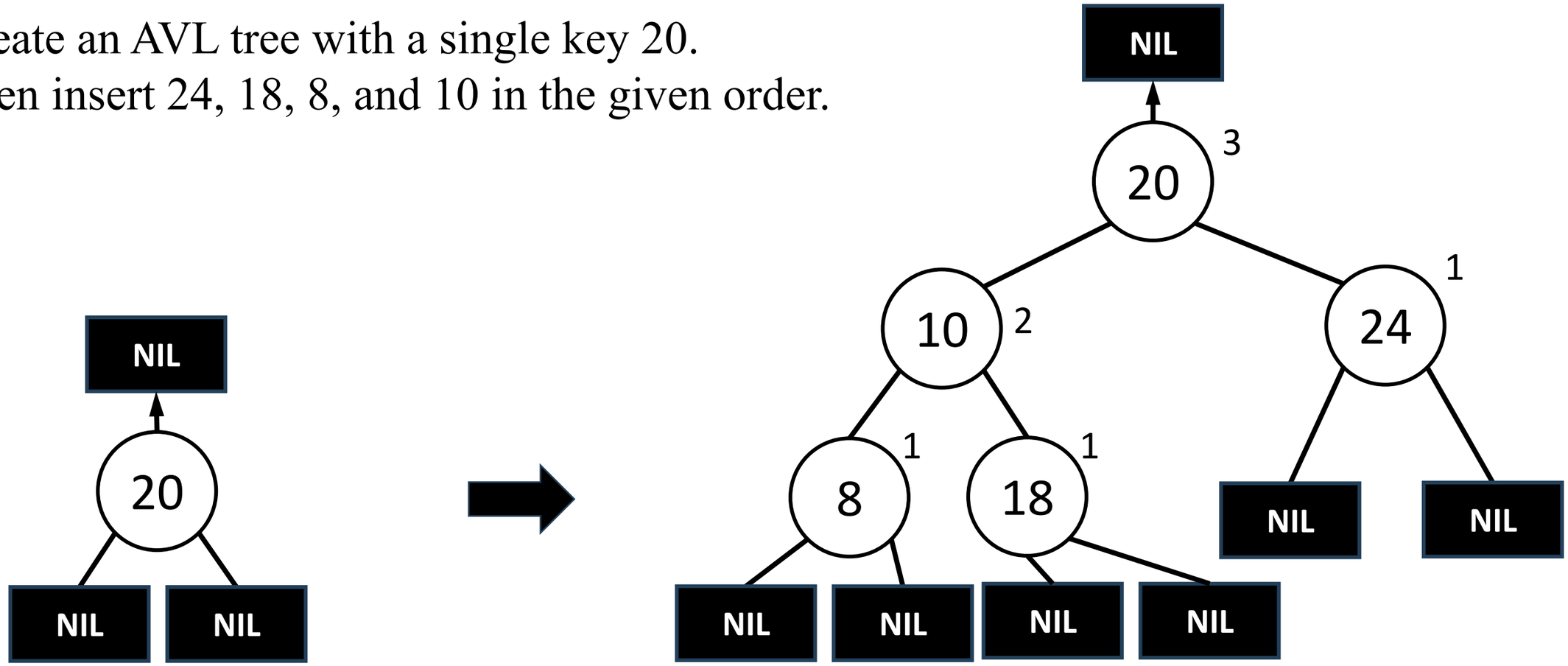
Next Class

We will discuss another very useful search tree known as **Trie** (pronounced ‘tree’ or ‘try’ by different people).

We will also discuss some graph algorithms for finding the **minimum spanning tree (MST)** of a graph and the **shortest paths** between a pair of nodes.

Today's Practice Problem

Create an AVL tree with a single key 20.
Then insert 24, 18, 8, and 10 in the given order.



Thank You